

DoubleLinkedList.c

****이중 연결 리스트(Double Linked List)**의 삽입, 삭제, 출력 기능을 구현한 C 언어 프로그램입니다.**

개요

이 파일은 ****이중 연결 리스트(Doubly Linked List)****를 정의하고, 삽입 및 삭제와 같은 기본 연산을 구현한 실습 예제입니다. 각 노드는 양쪽 방향의 링크를 가지며, 리스트 양방향으로 이동이 가능합니다.

주요 기능

◇ 리스트 초기화

- 더미 노드를 중심으로 **양방향 연결 구조**를 초기 설정합니다.

◇ 노드 삽입

- 주어진 노드 **before** 뒤에 새 노드를 삽입
- 양쪽 링크를 재조정하여 구조를 유지

```
void dinsert(DListNode *before, element data)
```

◇ 노드 삭제

- 특정 노드를 리스트에서 제거하고 연결 구조를 복원
- 메모리 누수를 방지하기 위해 **free** 사용

```
void ddelete(DListNode* head, DListNode* removed)
```

◇ 리스트 출력

- 헤드 노드 기준 오른쪽 방향으로 순회하며 노드 출력
- 출력 형태: **<- | |data| |->**

구조체 설명

```
typedef struct DListNode {
    element data;
    struct DListNode* llink;
    struct DListNode* rlink;
} DListNode;
```

- **llink**: 왼쪽 노드를 가리키는 포인터
- **rlink**: 오른쪽 노드를 가리키는 포인터
- **data**: 정수형 데이터 (element는 typedef로 정의됨)

🔗 컴파일 & 실행

```
# 컴파일
gcc -o DoubleLinkedList DoubleLinkedList.c

# 실행
./DoubleLinkedList
```

⚠ 참고: 일부 printf 문자열이 깨질 수 있으므로, **출력 인코딩 설정을 확인하세요.**

💡 학습 포인트

- ☒ 연결 리스트의 양방향 구조 이해
- ☒ 포인터 조작과 동적 메모리 할당
- ☒ 삭제 연산 시 링크 재조정 원리
- ☒ 구조체를 통한 노드 관리

🔍 예시: 이중 연결 구조

```
NULL <- [1] <-> [2] <-> [3] -> NULL
```

- 각 노드는 앞/뒤 모두 연결되어 있어 양방향 탐색이 가능함

⚠ 개선사항 제안

- 현재 코드에서 `strcpy(newnode->data, data);` 부분은 오류 가능성이 있음
`data`는 `int` 형이므로 `strcpy`가 아닌 직접 대입 필요:

```
newnode->data = data;
```